



**JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND TECHNOLOGY
SCHOOL OF ELECTRICAL, ELECTRONIC & INFORMATION ENGINEERING
INTERNAL INDUSTRIAL ATTACHMENT: TAFITTI PROJECT**

INDUSTRIAL ATTACHMENT REPORT

STUDENT'S NAME: ALLAN ALALA

REG. NUMBER: ENE212-0265/2018

COURSE: BSC. E.C.E 5.1

**PROJECT TITLE: MODULAR DESIGN OF SMART GREENHOUSE MONITORING
AND CONTROL SYSTEM PAIRED TO A LONG-RANGE COMMUNICATION
ARCHITECTURE.**

SUPERVISOR: ENG. S. N. MANEGENE

SUPPORT TECHNICIANS: ENG. C. OKETCH

ENG. MOSES

PROJECT PERIOD: JUNE TO AUGUST 2026

PRESENTED TO: ENG. E. W. MUREU

ASSESSING LECTURER, DEPARTMENT OF EEE, SEEIE, JKUAT

TECHNICAL PROJECT REPORT

Design and Implementation of a **Smart Greenhouse Monitoring and Control System**

With GSM SMS Alerts and Long-Range
LoRa Communication

Phases Covered:

Phase 1 – GSM-based SMS Alert System

Phase 2 – LoRa Long-Range Communication (Raspberry Pi 3 → ESP32-C6)

Submitted by Allan Alala – ENE212-0265/2018

Date: 18th September 2026

1. Abstract

This report presents the design, development, and implementation of a smart greenhouse monitoring and control system. The system collects environmental data from multiple sensors and monitors actuator states all using a lightweight Arduino Uno R3 microcontroller, then relays critical information about the sensor readings and actuator states to the farmer via a communication channel of choice. In the first phase, a GSM module was used to deliver SMS alerts. In the second phase, the GSM module was replaced with a LoRa (Long Range) communication system to enable reliable data transmission over long distances without reliance on cellular networks. The LoRa receiver was initially configured on a Raspberry Pi 3 and later migrated to an ESP32-C6 microcontroller to explore different platform capabilities. Communication between the Arduino-based transmitter and the receiver was successfully verified. The report covers system architecture, hardware selection, firmware development approach, testing outcomes, challenges encountered, and recommendations for future enhancements.

2. Introduction

Modern agriculture faces increasing pressure to improve yields while conserving resources. Greenhouses offer controlled environments that protect crops from adverse weather conditions and pests, yet they still require continuous monitoring of temperature, humidity, soil moisture, and other parameters. Manual monitoring is labour-intensive and prone to delayed responses. An automated system that uptakes environmental conditions around the greenhouse, processes the climatic data received from the sensors, controls actuators (such as irrigation pumps, fans, or vents) to apply corrective actions within the greenhouse, and promptly notifies the farmer on the state of the greenhouse, is one such system that would come in handy in reliable, efficient management of sustainable farming. This system enables timely interventions and optimises resource use.

This project addresses these needs through a two-phase implementation. Phase 1 focuses on a GSM-based SMS alert system for immediate farmer notification while the second phase extends the system with LoRa technology to support long-range, low-power communication suitable for remote or rural greenhouse sites where cellular coverage may be unreliable or costly.

3. Objectives

Primary objective:

To monitor and control the atmosphere in the greenhouse by measuring five different parameters of the greenhouse (temperature, relative humidity, light intensity, carbon dioxide, and soil moisture) automatically and manually.

The sub-objectives of the project were:

1. Design and implement a sensor and actuator data acquisition system for greenhouse environmental monitoring.
2. To design and develop a functional hardware prototype of a greenhouse model integrated with a power supply system, sensors, actuators and other passive components.

3. Develop a GSM-based SMS alert mechanism (Phase 1) to notify the farmer of critical sensor readings and actuator states.
4. Replace the GSM module with a LoRa communication link for long-range, low-power data transmission (Phase 2).
5. Configure the LoRa receiver first on a Raspberry Pi 3 and subsequently on an ESP32-C6 to evaluate different microcontroller platforms.
6. Verify end-to-end communication and alert delivery over the LoRa link.

4. System Overview

The overall system consists of a sensor/actuator node located inside or near the greenhouse and a remote monitoring/alert endpoint accessible to the farmer. Environmental parameters and actuator status are sampled by a microcontroller, processed according to predefined thresholds, and transmitted via the selected communication medium.

4.1 Phase 1 – Greenhouse Design with GSM SMS Alert System

In the first phase, an Arduino-compatible microcontroller interfaced with environmental sensors and actuators. A GSM module particularly the SIM800L model was connected to the microcontroller. When sensor readings exceeded or fell below set thresholds, or when actuator states changed, the system composed concise SMS messages and transmitted them to the farmer's mobile number. This approach provided immediate, cellular-network-based alerts without requiring a permanent internet connection at the greenhouse.

Parameters to be controlled:

- Light intensity
- Temperature & Humidity
- Soil moisture content & pH/nutrients levels
- Air quality/Carbon dioxide concentration
- Tank water level (parameter on the secondary layer)

4.2 Phase 2 – LoRa Long-Range Communication

The second phase replaced the GSM module with a LoRa transceiver pair to achieve greater range and reduced operational cost (no SIM card or airtime required). The transmitter side remained interfaced with the Arduino microcontroller that handles sensors and actuators. The receiver side was initially implemented on a Raspberry Pi 3, which received LoRa packets, processed the payload, and could display or forward alerts. Later, the Raspberry Pi was replaced with an ESP32-C6 module to gain hands-on experience with a modern, low-power, Wi-Fi/Bluetooth-capable microcontroller and to simplify the receiver hardware footprint.

LoRa operates in the sub-GHz ISM bands (commonly 433 MHz, 868 MHz, or 915 MHz depending on region) and employs chirp spread spectrum modulation, enabling reliable communication over several kilometres in rural environments while consuming very little power. For this particular project, I used the 433MHz band as it is the bandwidth applied in Kenya.

5. Hardware Components

The list below comprises the principal hardware components/elements required for assembling of the greenhouse model and their operational specifications:

5.1 Microcontrollers

The microcontroller is considered the main element in the coordinator station whereby it is responsible for the greenhouse climate control using equipment such as heaters, sprinklers, fan, light, windows. It serves as the central coordinator station responsible for interpreting sensor inputs and orchestrating climate control mechanisms through the actuators.

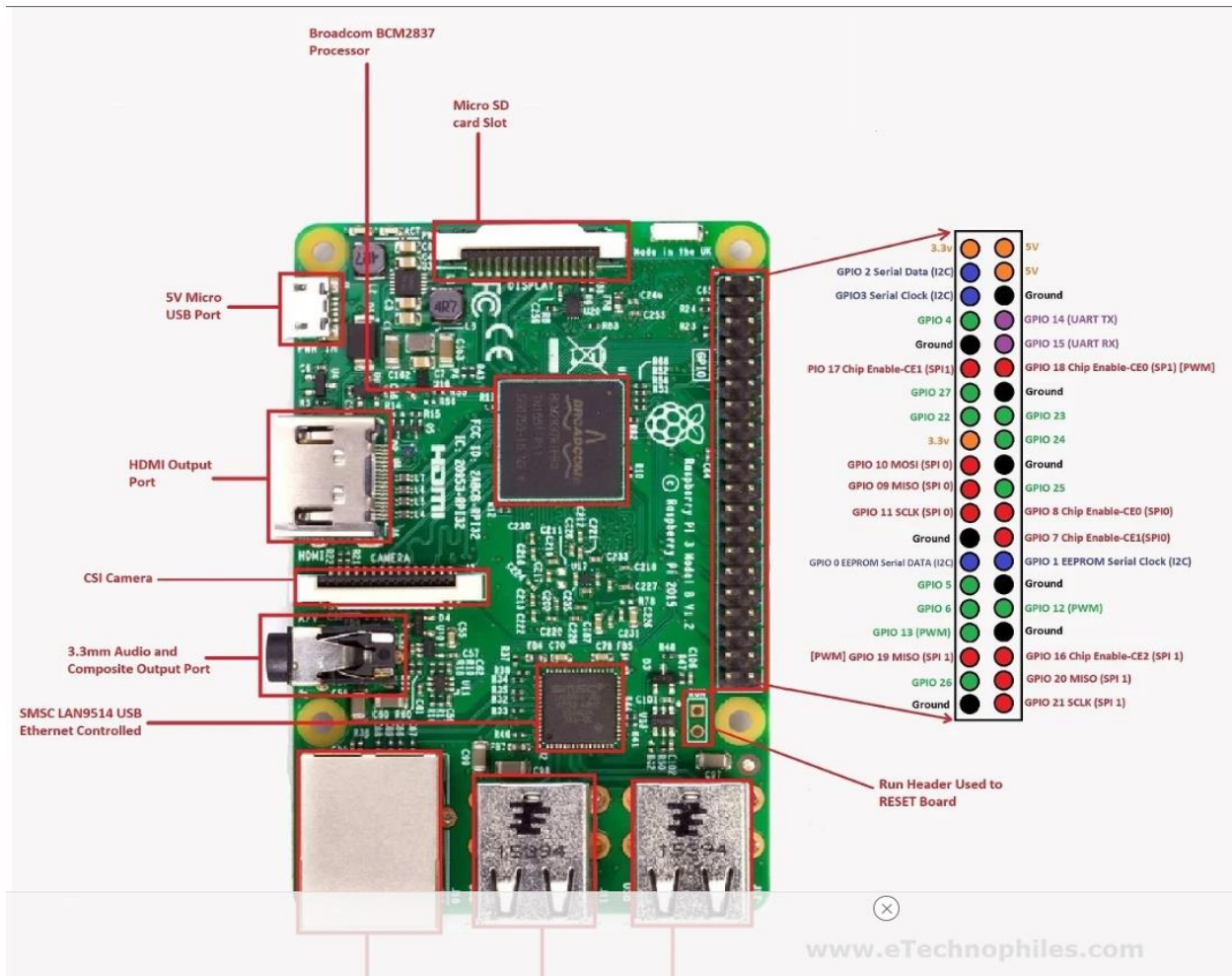
Arduino Uno R3

This is the main microcontroller unit (MCU) chosen for this project and is responsible for sensor reading, threshold logic, actuator control, and packet transmission. It was chosen for its user friendliness, I/O pins versatility, plug-and-play simplicity and the available vast library support from existing online communities among many other advantages.



Raspberry Pi 3 (Model B)

This board enjoys a relatively huge global user community backing, 40 pin GPIO header and a faster quad-core processor with low power consumption.



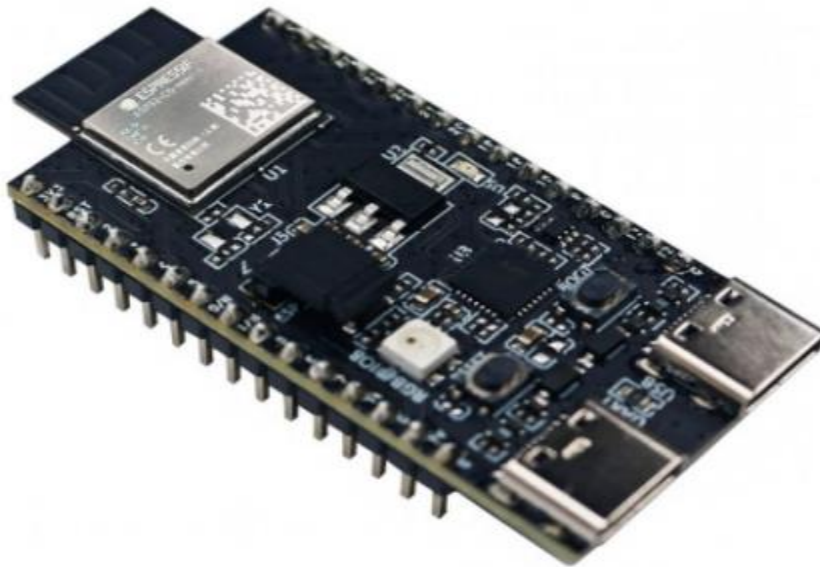
Connectivity: Wi-Fi 4 (2.4GHz) and Bluetooth 4.1

Power: I/O voltage – 5V (5-15W)

General-Purpose I/O pins: 40

ESP32 C6

The board is a single-core 32bit RISC architecture microcontroller that offers high processing power and integrated wireless communication features like **Wi-Fi**, **Bluetooth 5 (BLE 5)** and Zigbee radio communication onboard. Compatible with Arduino IDE. It easily integrates with platforms like Blynk for mobile control.



Specifications:

Memory	320kb ROM, 512kb HP SRAM, 16kb LP SRAM
I2C	A4-SDA, A5-SCL
SPI	D11 (COPI), D12 (CIPO), D13 (SCK), CS (Any GPIO)
Peripherals	UART, PWM, SPI, GPIOs

Connectivity: Wi-Fi 6 (2.4GHz) and Bluetooth 5.0

Power: I/O voltage – 3.3V

Digital I/O pins: 14

Analog I/P pins: 8

Merits:

Suitable for a wide range of IoT-based projects requiring wireless communication.

Cost effective and compact size.

Multiple hardware interfaces and communication protocols support.

Provides improved multitasking due to high power processing.

Demerit: Higher power consumption.

- Environmental sensors: Temperature and humidity sensor (e.g., DHT22 or SHT series), soil moisture sensor, light intensity sensor (LDR or photodiode), and optionally CO₂ or additional probes as required by the crop.
- Actuators: Relay-driven irrigation pump, ventilation fans, or motorised vents; status feedback may be obtained via current sensing or limit switches.
- Communication module (Phase 1): GSM module (SIM800L or equivalent) with SIM card.
- Communication module (Phase 2): LoRa transceiver module (e.g., SX1276/SX1278 based, such as RFM95 or equivalent) operating in the appropriate regional frequency band.

5.2 Sensor and Actuator Node (Transmitter Side)

- **Data acquisition modules**

Sensors

MAJOR SENSORS

									
1. DHT11 (Temperature & Humidity Sensor)	2. DHT22 (Temperature & Humidity Sensor)	3. BMP220 (Pressure & Temperature Sensor)	4. MQ-2 (Gas Sensor)	5. MQ-135 (Air Quality Sensor)	6. PIR (Motion Sensor)	7. HC-SR04 (Ultrasonic Sensor)	8. IR Obstacle Avoidance Sensor	9. Flame Sensor	10. Soil Moisture Sensor
									
11. LDR (Light Dependent Resistor)	12. BH1750 (Light Intensity Sensor)	13. UV Sensor (Ultraviolet Sensor)	14. Rain Sensor	15. Hall Effect Sensor	16. Reed Switch Sensor	17. Tilt Switch Sensor	18. MPU-6050 (Accelerometer & Gyroscope)	19. Vibration Sensor	20. Sound Sensor
									
21. MAX30100 (Pulse Oximeter & Heart Rate Sensor)	22. GPS (GNSS Sensor)	23. HMC5883L (Compass Sensor)	24. QMC5883L (Magnetometer Sensor)	25. MS5611 (Barometric Pressure Sensor)	26. MH-Z19 (CO ₂ Sensor)	27. Formaldehyde Sensor	28. Ammonia Sensor	29. DS18B20 (Temperature Sensor)	30. Thermocouple (K-Type)
									
31. Water Flow Sensor	32. Turbidity Sensor	33. pH Sensor	34. EC (Conductivity) Sensor	35. TDS (Total Dissolved Solids) Sensor	36. SDS011 (PM2.5/PM10 Sensor)	37. Dust Sensor (GP2Y1010AU0F)	38. Smoke Sensor	39. Water Level Sensor	40. Capacitive Touch Sensor

1. DHT11 – Measures temperature and humidity.
2. DHT22 – Measures temperature and humidity with higher accuracy than DHT11.
3. LDR – Measures light intensity.
4. MQ-135 – Detects air pollution and harmful gases. (CO₂ for my case)
5. Water level sensor

Further components

- GSM Module (SIM800L)
- Relay board (8-way)
- Servo motor (5V SG90)
- Water pump (5V DC pump)
- Water pipes (flexible PVC)
- LEDs (RGB)
- Cooling fans
- Solenoid valve
- Push buttons
- SPST ON/OFF Switch
- Connecting /jumper wires
- LCD
- Breadboards

- Rechargeable LiPo battery
- Output terminals
- Soil container

- **Actuators**

8-way Relay

Actuation is achieved through multi-channel relay module that controls the switching of the peripheral devices such as the pumps and cooling fans.



Telemetry & Communication

Here, we consider factors such as power consumption, data transmission speed, communication range, environmental considerations, and the maximum number of devices that can be connected. I2C is an optimal choice for communication with multiple sensors using minimal wiring. RS232 excels in long-distance communication, whereas SPI is tailored for rapid and reliable data transfers.

Integrated Wireless Communication: With onboard Wi-Fi (2.4GHz) and Bluetooth 5.0, it eliminates the need for expensive external network modules. This directly supports seamless, bidirectional integration with the Blynk IoT platform for real-time remote control.

- **Wi-Fi connectivity**
- **GSM module**



- **LoRa Communication**

After demonstrating GSM module functionality in sending messages to the farmer's mobile phone upon greenhouse parameter changes or actuator states changes, we sought to replace the GSM module with a set of LoRa transceiver modules.

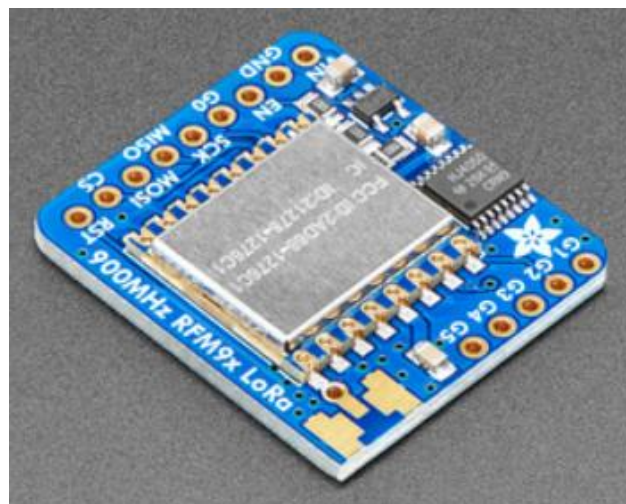
I settled on the RFM96 module which was readily available and cost effective for the project and configured one module to the Arduino MCU and the other to the Raspberry Pi. The module provides a bidirectional communication setup in which both can act exchange data as a receiver or transmitter.

HopeRF RFM95 / RFM96 / RFM98W

Pre-packaged breakout modules built around the SX127x die, with the RF matching network and shielding are already done for you. Extremely popular in hobbyist and early commercial IoT designs due to low cost and drop-in SPI footprint.

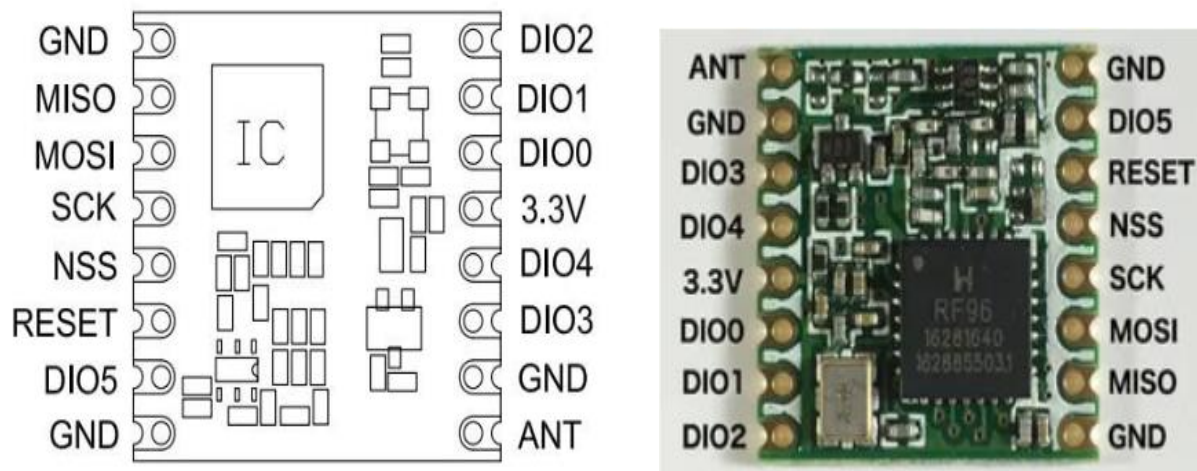


RFM95W



Building Blocks

- SX127x die (mounted internally)
- Castellated-edge module footprint for easy PCB reflows soldering
- Onboard impedance-matching network tuned to the module's target band
- SPI interface pins broken out (MOSI, MISO, SCK, NSS, DIO0-DIO5)



Limitations

- Still requires an external MCU (ESP32, STM32, AVR, etc.) to run any application logic
- Maximum transmit power capped around +20dBm
- Highly sensitive to poor PCB grounding or missing ground-plane cutouts under the antenna trace
- No onboard TCXO on cheaper variants — frequency drift over temperature

5.3 Receiver Side

- Initial implementation: Raspberry Pi 3 with compatible LoRa HAT or SPI-connected LoRa module. The Pi provided a convenient Linux environment for packet reception, logging, and potential further processing or display.
- Subsequent implementation: ESP32-C6 microcontroller with integrated or external LoRa radio. The ESP32-C6 was selected to reduce power consumption, physical size, and cost while retaining sufficient processing capability and offering built-in wireless connectivity options for future expansion.

6. Software Development

Firmware was developed iteratively for each platform without reliance on external scripts in the final documentation. The development approach emphasised modularity, clear separation of sensor acquisition, decision logic, and communication layers.

6.1 Arduino Transmitter Firmware

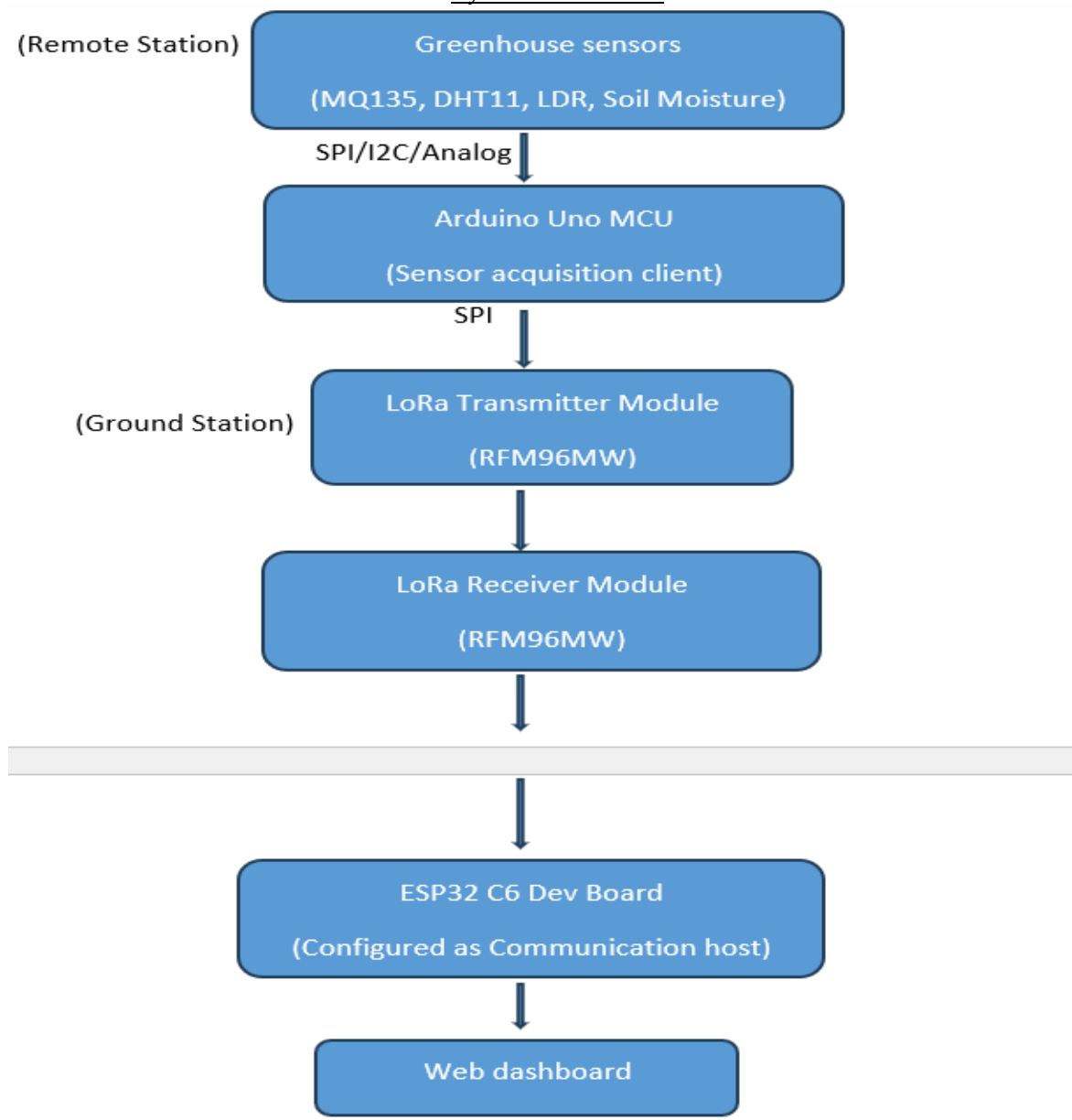
The Arduino firmware performed the following core functions:

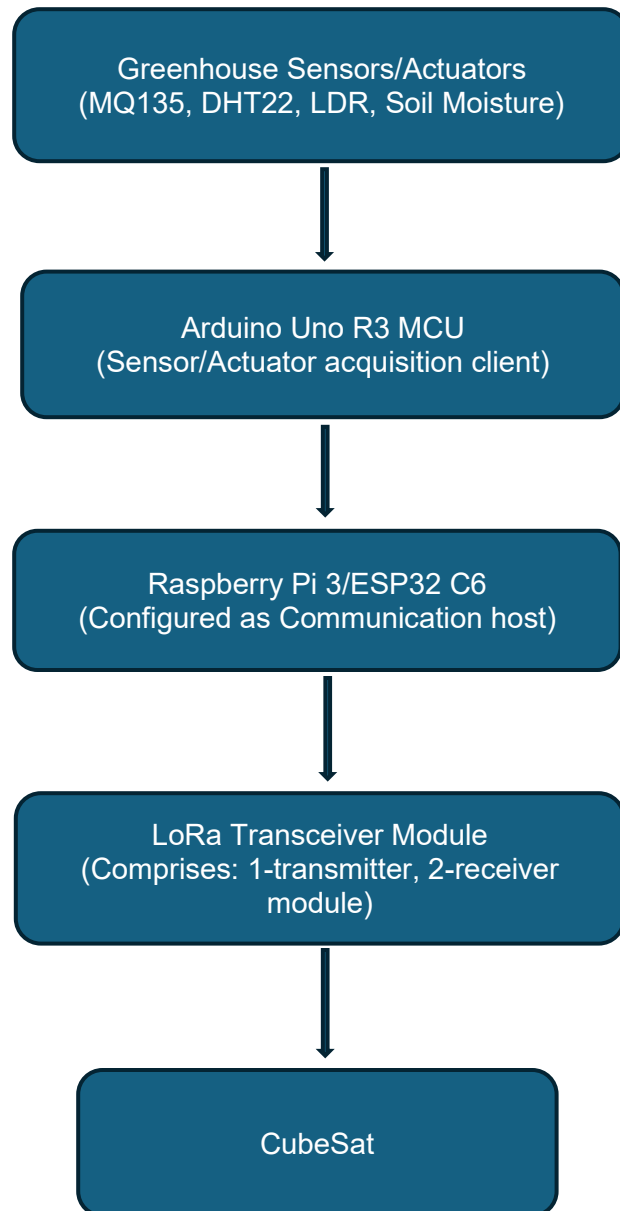
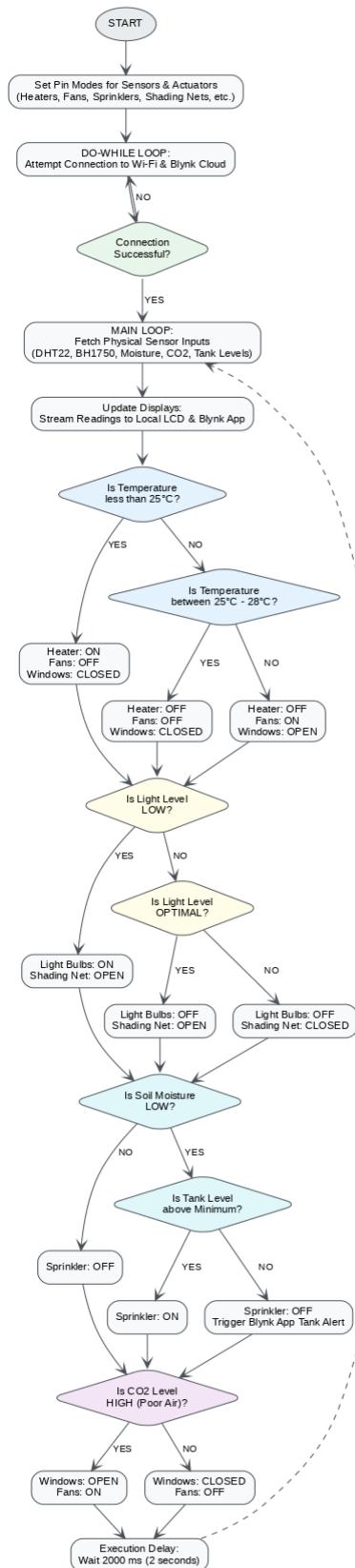
- Periodic sampling of all connected sensors at configurable intervals.
- Application of threshold rules to determine whether an alert condition existed.
- Control of actuators according to measured values or received commands (where bidirectional capability was implemented).
- Composition of a compact data payload containing sensor readings and actuator status.

- Transmission of the payload via the active communication module (GSM AT commands in Phase 1; LoRa radio driver in Phase 2).

Software Implementation

System Flowchart





Particular attention was paid to power management, reliable serial communication with the radio modules, and graceful handling of temporary sensor or radio failures.

6.2 Raspberry Pi Receiver (Initial LoRa Setup)

On the Raspberry Pi 3, a user-space program (or Python-based service) was developed to initialise the LoRa radio over SPI, listen continuously for incoming packets, validate packet integrity, parse the sensor/actuator payload, and present the information via console output, local logging, or a simple notification mechanism. Configuration of the LoRa parameters (frequency, spreading factor, bandwidth, coding rate) was performed to match the transmitter settings, ensuring reliable link establishment.

6.3 ESP32-C6 Receiver (Final LoRa Setup)

The migration to the ESP32-C6 involved porting the packet reception and parsing logic to the ESP-IDF or Arduino-ESP32 framework. The ESP32-C6 firmware initialised the LoRa radio, maintained a continuous receive loop, decoded incoming packets, and could optionally forward alerts via its native Wi-Fi or Bluetooth interfaces for integration with a smartphone or local dashboard. This platform demonstrated lower power consumption and a more compact form factor compared with the Raspberry Pi while still providing adequate processing resources for the application.

6.4 Software Tools

Arduino IDE: this is the official primary software tool used to write, compile and upload program sketches to the Arduino Uno R3 microcontroller. It allows installation, update and inclusion of external software libraries to add pre-written functions that simplify complex hardware interactions. It also provides a specialized text editor with C++ syntax structure to edit scripts. Additionally, it lets one select the correct board model and communication port for hardware interfacing.

Raspberry Pi Imager: this is the official downloaded software tool used to install the Raspberry Pi 3 Operating System (OS) onto the SD card we had mounted onto the board.

PuTTY: this was used to allow secure remote access to the Raspberry Pi 3's command line from my laptop using Secure Shell (SSH). It is a free, open-source terminal emulator and network communication client that lets you perform initial configurations and manage sessions by saving profiles.

Blynk: this is an interactive platform used for building web dashboard by providing a drag-and-drop widget configuration, multilayered data filtering and hardware-to-cloud routing.

Real VNC: this software came in handy by providing remote desktop access to the Raspberry Pi 3's OS enabling us to view and interact with the graphical user interface (GUI) without the need for an external monitor, keyboard or mouse. Since it is a paid service, we only enjoyed its services for a few days while taking advantage of the free trial package.

Circuit Design Using Software Environment

Proteus: this is a software product of Proteus Design Suite that provides a virtual workspace for designing, testing, and troubleshooting electronic circuits and microcontroller-based systems

before building physical prototypes by allowing schematic captures, virtual instruments, PCB layout designs and circuit simulations.

Wokwi: this is an online platform used for virtual prototyping, testing and validating microcontroller code, circuits, and IoT projects without physical hardware.
EasyEDA

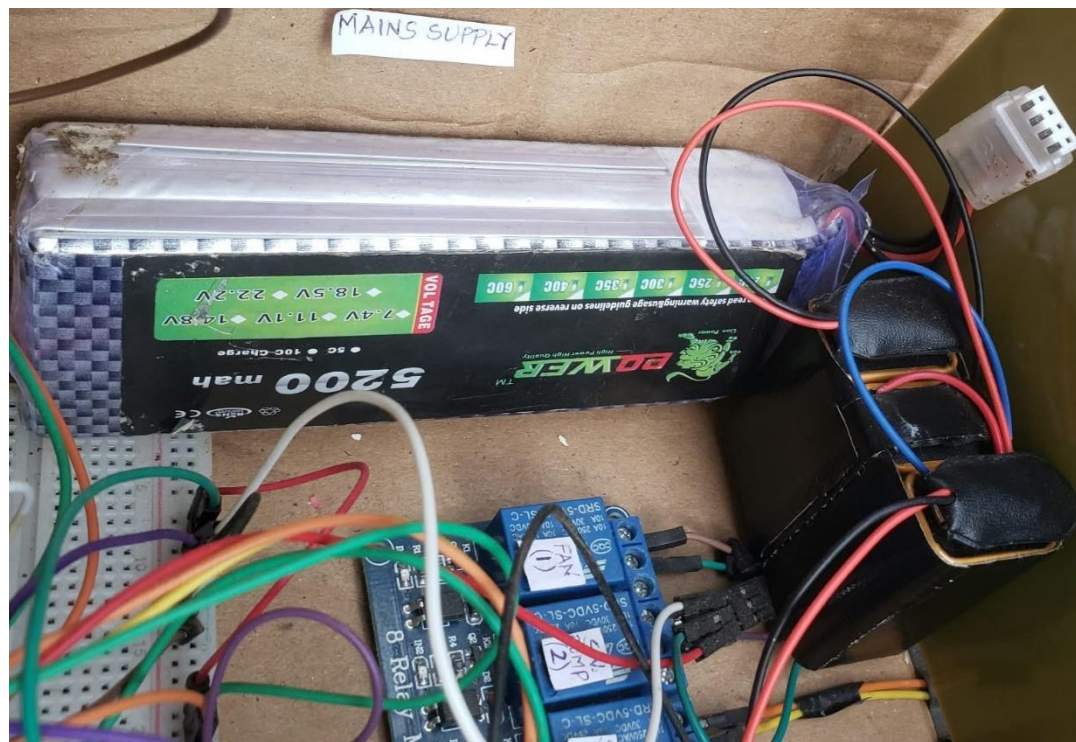
7. Implementation and Testing (Methodology)

Implementation proceeded in stages. Sensor and actuator interfacing was validated first on the Arduino platform using serial monitoring. GSM SMS functionality was then verified by triggering threshold events and confirming message delivery on the farmer's handset. After successful Phase 1 validation, the GSM module was replaced by the LoRa transceiver on the transmitter side.

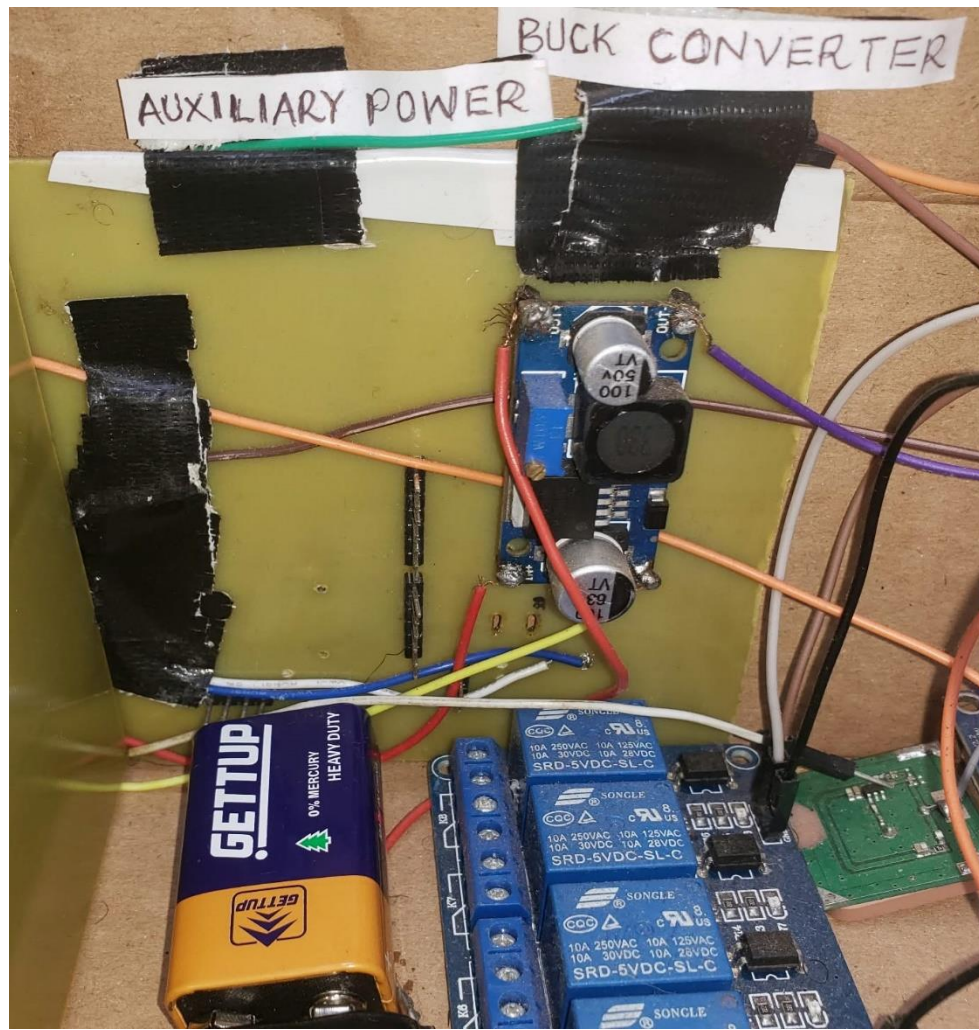
7.1 Power Supply and management design

The whole circuit was powered by a combination a rechargeable and a pair of non rechargeable 9V Alkaline Gettop batteries wired to deliver the requisite circuit current demands.

This 11.1V Power 5200mAh LiPo battery was applied as the main circuit supply for the circuit neatly wired to the power terminal from where the various components can tap the power from.



Due to the varying power requirements of the components of the circuit, it was prudent to design a buck converter architecture to step down the voltage to lower levels required by the specific components, for instance, the relay module requires strictly 5V to perform the actuation action of the different terminal devices.



7.2 Functional Logic design

- Climatic control:** If the DHT22 detects the internal temperature rising above or below 26°C, the relay triggers a cooling fan or the window curtain rail actuator to apply the corrective response to the prevailing temperature condition. DHT22 was chosen over the lower-grade DHT11 due to its superior operating range (-40°C to 80°C, 0–100% RH) and higher precision ($\pm 0.5^\circ\text{C}$ temperature accuracy). It communicates via a single-wire digital bus, ensuring data integrity over longer cable runs within the greenhouse.

Level	Action
Very low (Below 15°C)	Heater ON, Fans OFF, Windows close.
Low (15 - 24)°C	Heater ON, Fans OFF, Windows close
Optimal (25 - 28)°C	Heater OFF, Fans OFF, Windows close.
High (29 - 35)°C	Heater OFF, Fans ON, Windows open.
Very high (Above 36°C)	Heater OFF, Fans ON, Windows open.



- **Irrigation control:** If the analog signal from the soil moisture sensor drops below a certain threshold (e.g., bone-dry soil), the microcontroller closes the relay circuit to activate a 12V DC water pump which then pumps the water from the storage tank to the sprinkler lines.

Threshold conditions

Level	Action
Low (Below threshold, say $H < 60\%$)	Pump/Sprinkler/Dripline ON
Optimal (Between 60% and 70%)	Pump/Sprinkler/Dripline OFF
High (Above threshold, say $H > 70\%$)	Pump/Sprinkler/Dripline OFF

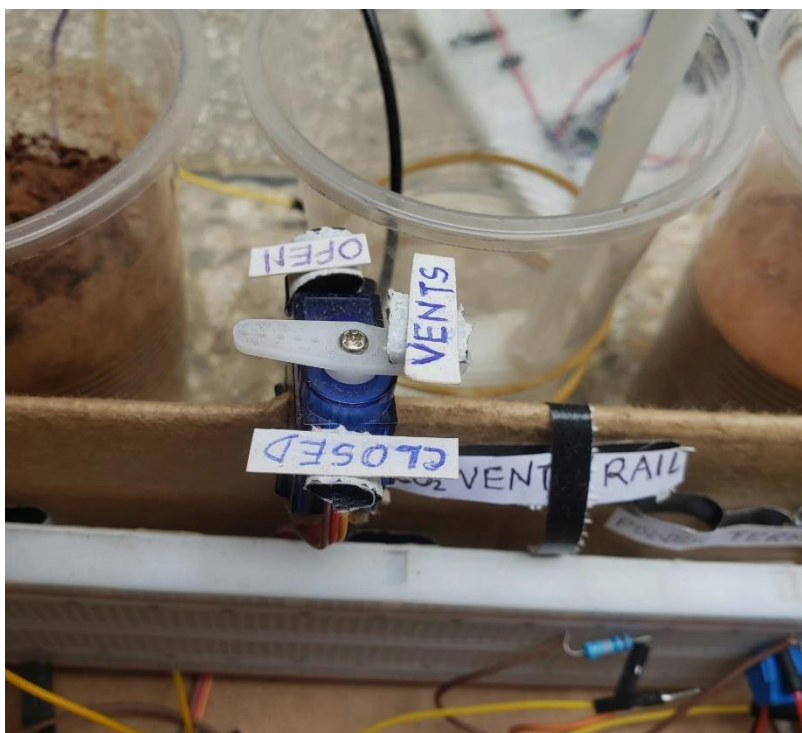


- **Lighting control:** If lighting drops such as in the case of heavily cloudy days or when darkness sets in, Light dependent resistors (LDRs) are switched on to control grow lights to ensure your crops get a consistent schedule. LED grow lights in this project are represented by an LED that emits blue light.

Threshold conditions

Level	Action
Low	LED grow lights ON, Shading open(window rails up)
Optimal	LED grow lights OFF, Shading open (window rails up)
High	LED grow lights OFF, Shading close (window rails down)

- **Air quality control:** If after long hours of closer of the windows/ventilation spaces result into a significantly lower levels of CO₂ concentration, say for instance overnight, such information is relayed to the microcontroller by the MQ135 sensor resulting into the invocation of a corrective mechanism by the actuators in this case the CO₂ vents opening or cooling fans and the motor controlling the window rails swing into action to allow free flowing of air in and out of the greenhouse in order to bring the concentration of the important gas to the natural levels (to the extent of natural composition in air).



CO₂ vents opening/closing action modelled using SG90 servo movement.

Level	Action
Low (High CO ₂ concentration)	Vents open, Fans ON
Optimal	Vents open, Fans OFF
High (Low CO ₂ concentration)	Vents closed, Fans OFF

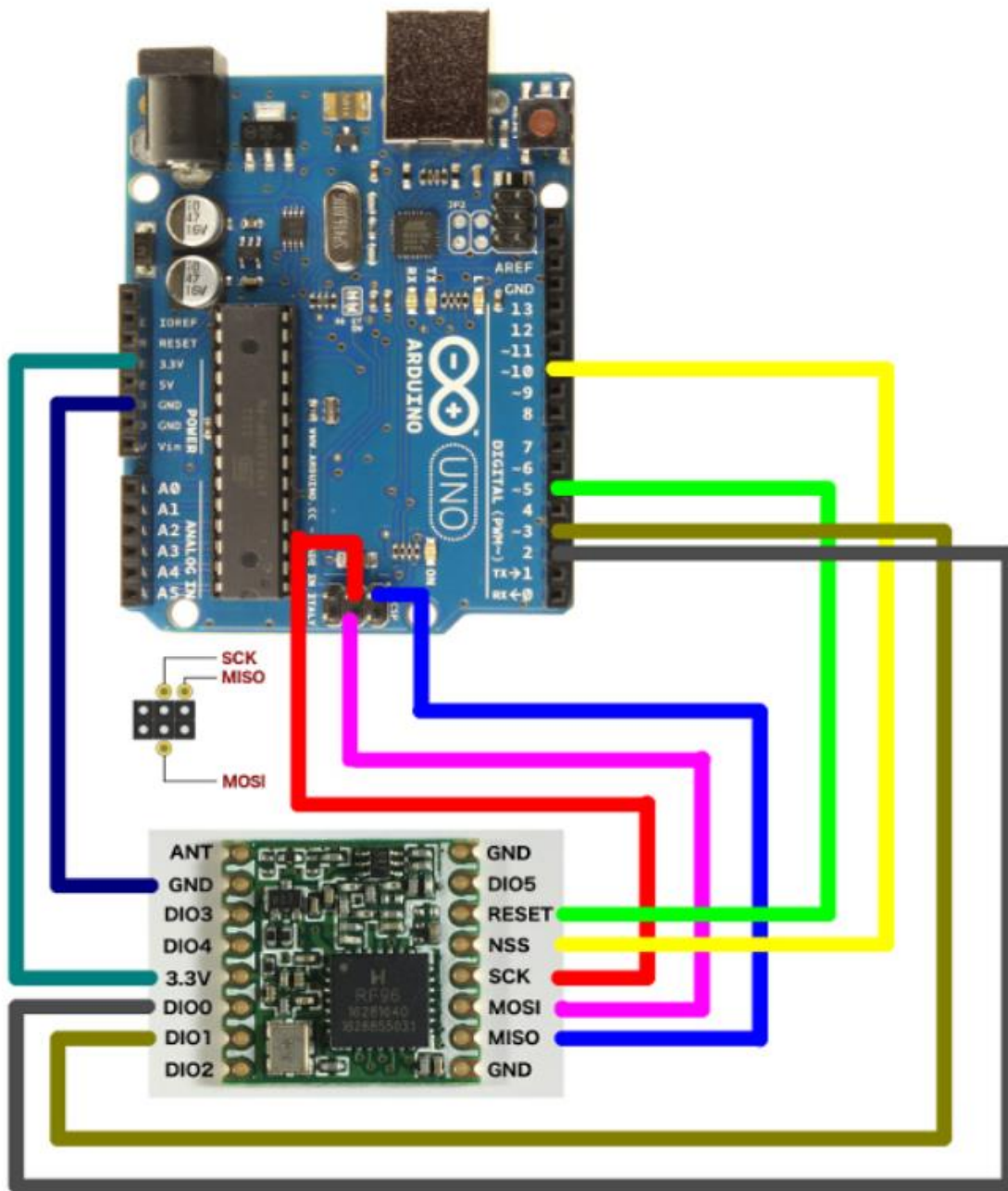
- **Water Level Monitoring:** An analog water level sensor tracks remaining capacity in the storage tank. It maps water depth to an analog output voltage range (0V - 3.85V), protecting the irrigation pump from burning out if water runs low.

By implementing these automated control loops, the system effectively minimizes human error, optimizes resource usage, and cuts down on manual labor overhead.

For the LoRa link, the receiver was first brought online on the Raspberry Pi 3. Frequency, spreading factor, and other radio parameters were aligned between transmitter and receiver. Packet reception was confirmed through console logging of received payloads. Once stable communication was established, the Raspberry Pi was replaced by the ESP32-C6. The same

radio parameters were retained, and end-to-end alert delivery was re-verified. Tests included line-of-sight and non-line-of-sight scenarios at increasing distances to characterise practical range under typical greenhouse and farm conditions.

Arduino to LoRa Configuration



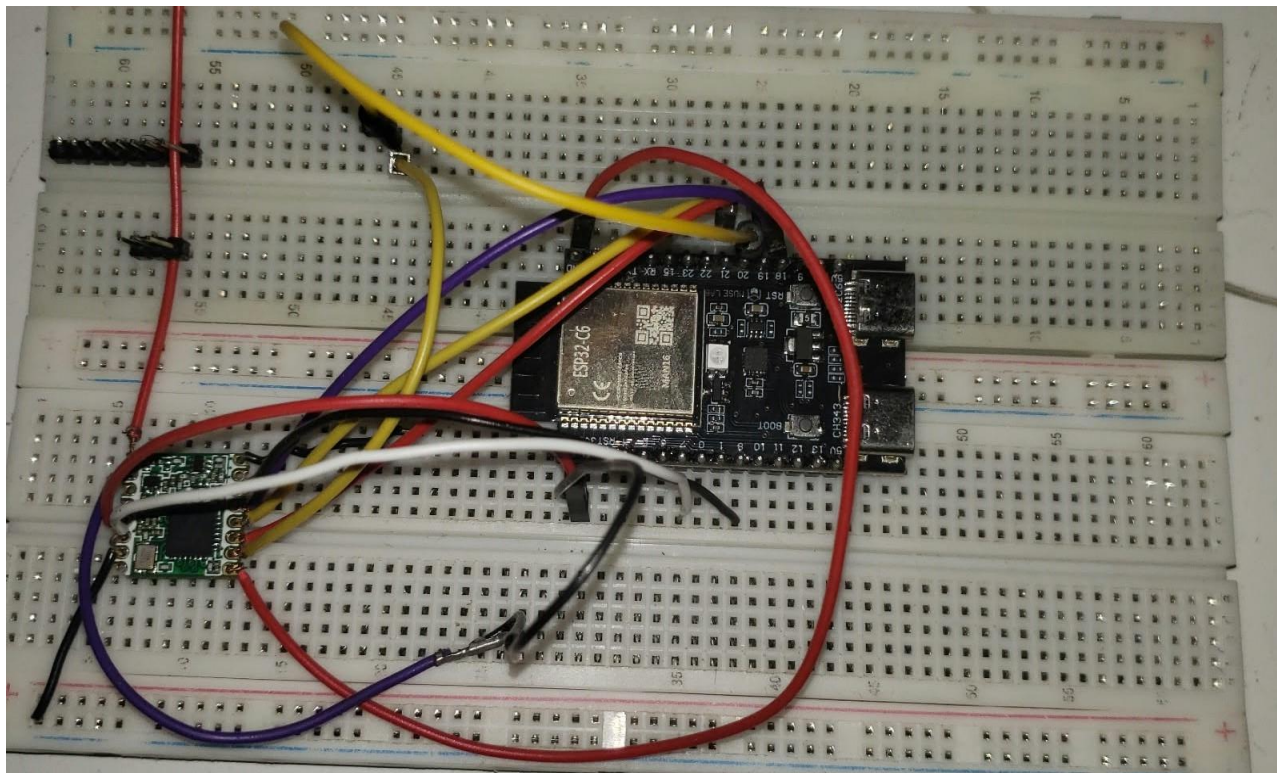
Raspberry Pi to LoRa Configuration

Hardware configuration:

Raspberry Pi 3 Pin	LoRa Module Pin	Physical Pin Number
3.3V	VCC	Pin 1 or 17
Ground	GND	Pin 6, 9 or 14

SPIO CE0 (GPIO 8)	NSS / CS	Pin 24
SPIO MISO (GPIO 9)	MISO	Pin 21
SPIO MOSI (GPIO 10)	MOSI	Pin 19
SPIO SCLK (GPIO 11)	SCK	Pin 23
GPIO 4 (Customizable)	DIO0/IRQ	Pin 7
GPIO 25 (Customizable)	RST	Pin 22

ESP32 C6 to LoRa Configuration



Sensor collector (Arduino Uno Rev3) configuration:

Program the Arduino MCU to poll the sensors, format the data into a compressed, delimited string (to save bandwidth), and print it straight to the serial port.

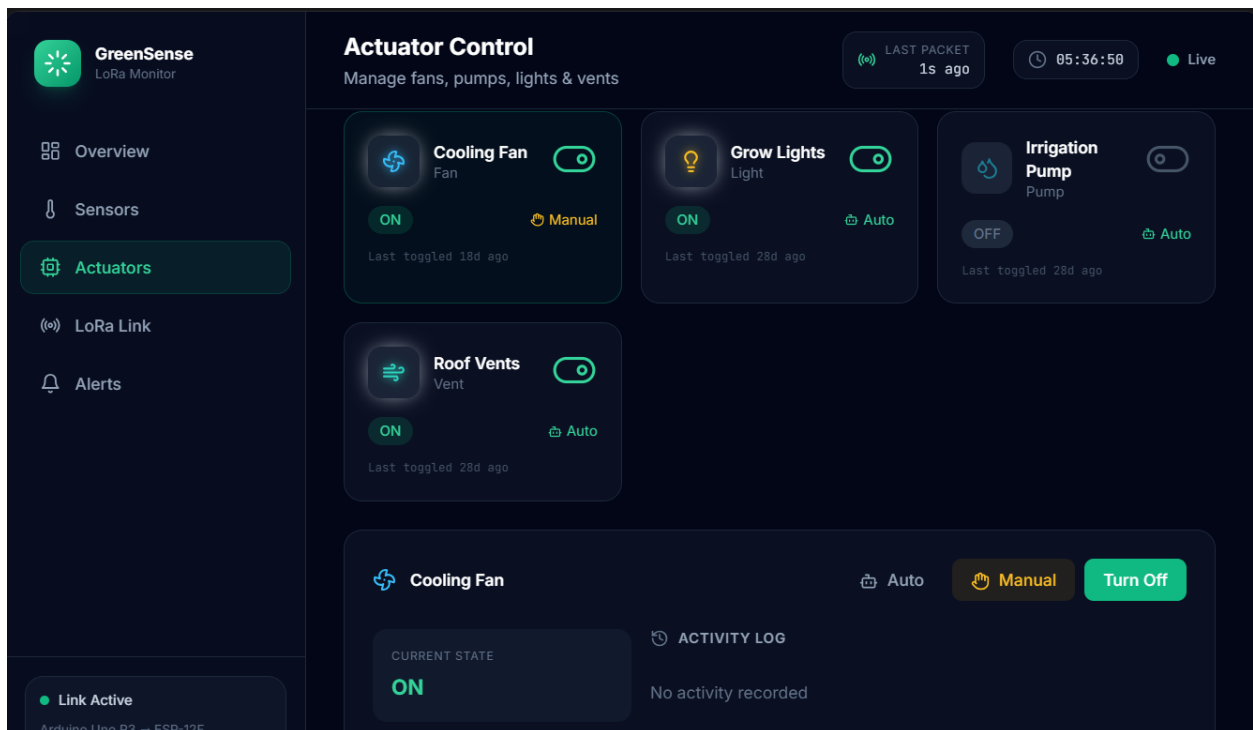
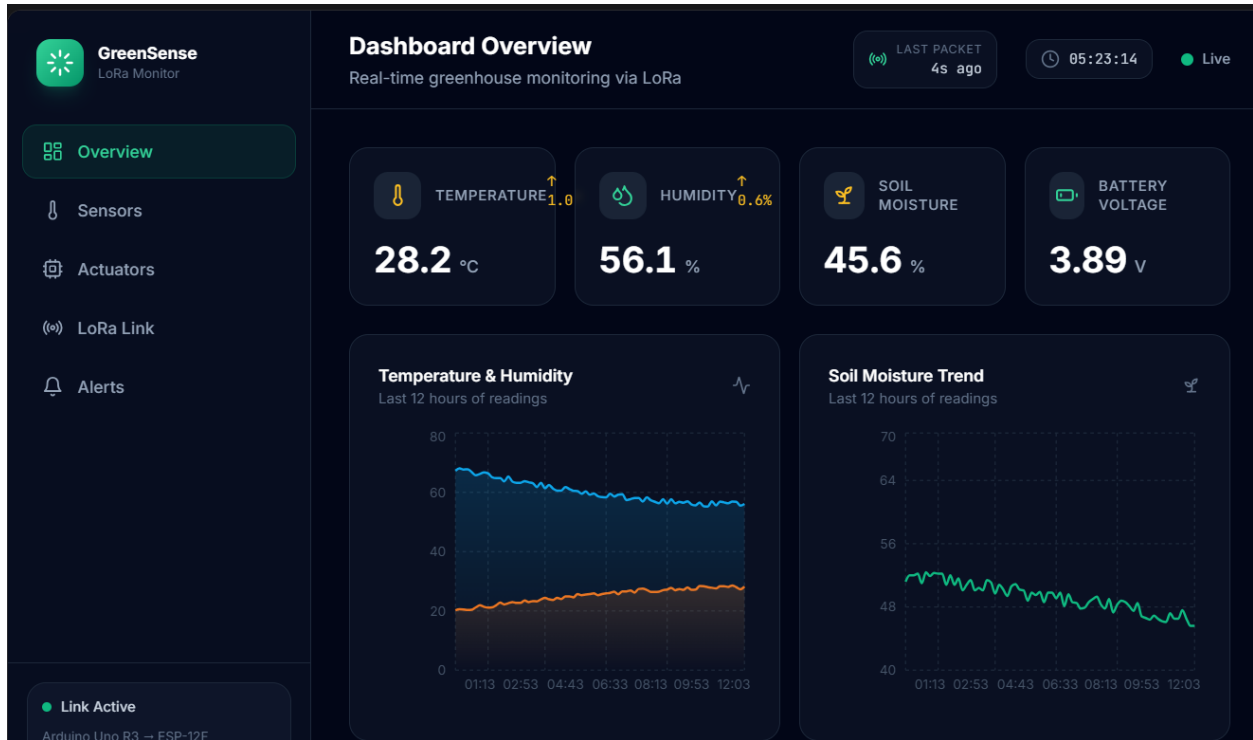
Communication host (Raspberry Pi 3) configuration:

The Raspberry Pi 3 handles incoming serial streams, manages data queues, packs payloads, and controls the LoRa transmitter.

Data buffering script

Serves to save readings in a local database on the Pi until an overhead satellite pass window opens. ANT to a tuned 433 MHz wire antenna or an external improvised antenna.

Web Dashboard





8. Problems Encountered and Solutions

I was able to draw on the previous experience from the early interaction with series of programming classes for the GUI, MCU-based system design for our serial communication and hardware level commands, power systems from earlier courses for the battery-life analysis, and Integrated Circuits for the housing. However, there were still many areas that I needed to learn entirely new skills. The LoRa and GPS chipset are controlled using AT-commands, which I had not been previously exposed to. I also needed to learn the fundamentals of the Raspberry Pi 3 in order to initialize communication ports, run programs on start-up, and calibrate the web dashboard. This included the initial headless configuration procedures. All in all, I together with the other group members, were able to combine these different knowledge sets and build a fully functioning smart greenhouse monitoring & control system using all of the above functionality. Several practical challenges arose during development and integration:

1. Overall power management: this was perhaps an everyday problem since the assembly of the prototype components into place. The continued testing of the sensor/actuator systems resulted into draining of the Lithium-ion batteries, which constantly necessitated replacement. Also, the actuators such as the relay module and induction motor used to model the fan had different power requirements prompting providing individual supply voltages wired to a common breadboard terminal.
2. Initial Raspberry Pi 3 setup and subsequent LoRa configuration: this was perhaps the most frustrating and challenging task in the entire project. Since we lacked a dedicated display and keyboard, the viable approach for the initial setup was the headless setup which involved using a Raspberry Pi Imager to write an OS to the board via an SD card

and consequently booting. The board had a lot of challenges enabling SSH and pre-loading Wi-Fi during OS customization.

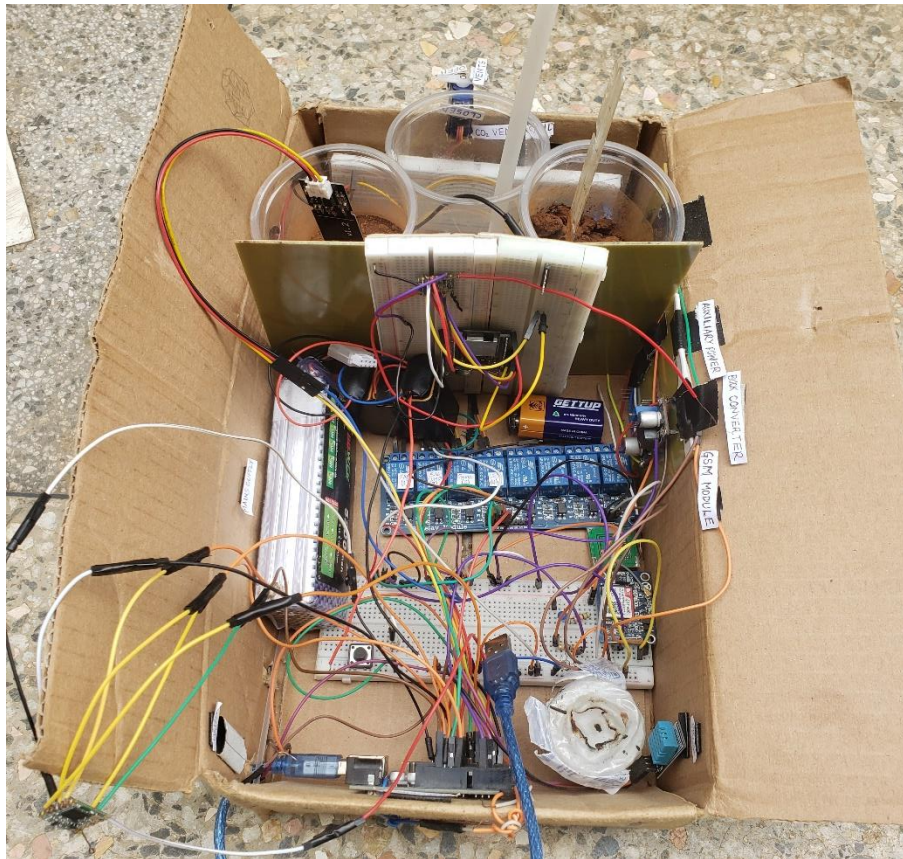
3. GSM module reliability and power: The GSM module exhibited sensitivity to supply voltage dips during transmission. This was mitigated by providing a dedicated, adequately rated power supply by tapping its supply from an LM2596 buck converter to provide the requisite voltage level.
4. SIM card and network registration delays: Occasional delays in network registration were observed. Firmware was adjusted to include retry logic and status checking before attempting to send SMS messages.
5. LoRa parameter mismatch: Initial packet loss was caused by mismatched spreading factor or bandwidth settings between the Arduino transmitter and the receiver. Systematic documentation of radio configuration and a shared parameter set resolved the issue.
6. SPI and pin conflicts on Raspberry Pi: Correct GPIO and SPI device assignment required careful reference to the Pi pinout and the LoRa HAT documentation. Once the correct chip-select and interrupt pins were identified, communication stabilised.
7. Migration from Raspberry Pi to ESP32-C6: Differences in library APIs and peripheral initialisation sequences necessitated a partial rewrite of the radio driver layer. Incremental testing of SPI communication and then full packet reception helped isolate issues quickly.
8. Antenna placement and range variability: since I was using an improvised antenna (coiled 12cm copper wire), it was difficult to deduce the precise performance behaviour but upon comparing that with an OEM antenna drawn from the previous GSM module, the performance varied with antenna orientation and surrounding obstacles. Optimal antenna placement (elevated, clear of metal structures) and use of appropriate antennas for the chosen frequency band improved link reliability.

9. Results and Discussion

Phase 1 of the project successfully demonstrated the real-time operation of the system autonomously by using the environmental conditions and that sensor data and actuator status could be reliably packaged into SMS alerts and delivered to a mobile handset. By altering the climatic conditions around the prototype such as switching the soil containers or fanning around the DHT22 & MQ135 sensors, the supervisor was able to observe the system responsiveness and the invocation of the actuator devices to respond to the parameter changes. The farmer received timely notifications whenever monitored parameters crossed defined thresholds.

In Phase 2, LoRa communication was established between the Arduino transmitter and both receiver platforms. The initial setup of the receiver node was implemented by a Raspberry Pi which was not very successful since the board had a lot of unreliability issues ranging from power connection to network connectivity. Consequently, we switched to an ESP32-C6 microcontroller which offered successful communication pairing with much ease. Packets containing sensor readings and actuator states were received consistently once radio parameters were aligned. The transition from Raspberry Pi 3 to ESP32-C6 confirmed that a more compact, lower-power microcontroller could fulfil the receiver role while opening pathways for future Wi-Fi or Bluetooth integration.

Overall, the dual-phase approach validated both cellular and long-range radio solutions. GSM proved convenient where cellular coverage is good; LoRa offered a cost-effective, infrastructure-independent alternative better suited to remote greenhouse locations.



10. Conclusion

A complete smart greenhouse monitoring and alerting system was designed and implemented in two complementary phases. The GSM-based Phase 1 delivered reliable SMS alerts to the farmer. The LoRa-based Phase 2 successfully replaced cellular dependency with a long-range, low-power radio link. Receiver functionality was demonstrated first on a Raspberry Pi 3 and subsequently on an ESP32-C6, confirming platform flexibility. End-to-end communication and alert delivery over LoRa were verified. The project achieved its stated objectives and provided a solid foundation for further refinement and field deployment.

11. Recommendations and Future Work

- It would be necessary to introduce a battery management system (BMS) for continuous monitoring of battery health, state of charge, safety and communication so as to ensure reliable, uninterrupted and safe delivery of power to the sensors and actuators.
- Add bidirectional communication so that the farmer can send control commands for instance force irrigation back to the greenhouse node.

- Integrate a simple web or mobile dashboard on the ESP32-C6 (via Wi-Fi) for real-time visualisation of sensor trends.
- Implement low-power sleep modes on both transmitter and receiver to extend battery life for off-grid installations.
- Conduct extended field trials under realistic greenhouse conditions to characterise range, packet loss, and power consumption over seasonal variations.
- Explore multi-node LoRa networks (multiple greenhouses reporting to a single gateway) for larger agricultural operations.
- Introduce a Machine Learning Model to monitor and act on pests and diseases on the plants grown in the greenhouse in order to eliminate the need for human labour in inspecting for disease attacks on the crops. This can be achieved by use of a pre-trained machine language (ML)

12. References and Supporting Material

Hardware datasheets and library documentation for the Arduino platform, SIM800L (or equivalent) GSM module, SX127x LoRa transceivers, Raspberry Pi 3, and ESP32-C6 were consulted throughout development. Specific part numbers and regional frequency regulations should be confirmed for any future commercial deployment.

No source-code listings are included in this report, in accordance with the project documentation requirements. Firmware design principles and functional behaviour have been described in the preceding sections.

1. ESP documentation: https://documentation.espressif.com/esp32-c6_datasheet_en.pdf
2. Raspberry Pi 3 documentation: <https://www.etechnophiles.com/raspberry-pi-3-gpio-pinout-pin-diagram-and-specs-in-detail-model-b/>
3. https://www.academia.edu/35483466/Greenhouse_Monitoring_System_Using_GSM?email_work_card=reading-history

— *End of Report* —